

# Frontend + AI + Hackathon Resource Guide

A practical reference vault compiled from the reels/captions you supplied. Built for direct use during frontend development, rapid prototyping, hackathons, portfolio work, and AI-assisted coding.

How to use this PDF: Treat it as a field manual. When you need a UI component, animation, layout pattern, AI workflow, freelancing platform, or automation, jump to the relevant section and use the reference links. The notes deliberately focus on what to use, when to use it, and what to avoid.

Important: A few reel claims are marketing-heavy or insufficiently specific. Those are explicitly marked as VERIFY instead of being presented as facts. The goal is a reliable development reference, not an Instagram-caption museum.

## Quick Navigation

- 1 — UI components and design styles
- 2 — Frontend libraries
- 3 — Scroll and motion engineering
- 4 — Layout patterns
- 5 — AI frontend design/coding stack
- 6 — Automated AI resume workflow
- 7 — Problem statements and freelancing
- 8 — Claude prompting claims
- 9 — Antigravity / agentic coding
- 10 — Hackathon implementation playbook
- 11 — Quick-reference decision table

# 1. UI Components and Design Styles

These resources help when the bottleneck is not your logic but the amount of time it takes to make a frontend look good. Use them for inspiration, copy/paste starting points, and visual direction. Always check the actual license/usage terms for the specific asset you copy.

| Resource                | Reference                                                                               | Use it for                                                                                                                                                             |
|-------------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ulverse                 | <a href="https://uiverse.io/">https://uiverse.io/</a>                                   | Open-source/community UI elements. Useful for buttons, cards, loaders, inputs, switches, glass-style elements, Tailwind/React examples and Figma-oriented exploration. |
| Ulverse GitHub / Galaxy | <a href="https://github.com/uiverse-io/galaxy">https://github.com/uiverse-io/galaxy</a> | Useful when you want to inspect or reuse the underlying open-source collection rather than only browsing the website.                                                  |
| Google Stitch           | <a href="https://stitch.withgoogle.com/">https://stitch.withgoogle.com/</a>             | AI-assisted UI generation/prototyping. Useful for turning a product idea or prompt into interface concepts and then moving toward implementation.                      |

## Design styles from the reels

| Style         | Visual language                                                                                      | Good use                                                                                                             |
|---------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Glassmorphism | Translucent surfaces, backdrop blur, borders, depth, layered backgrounds.                            | Dashboards, landing pages, media/product UIs. Keep contrast high; do not turn every card into a frosted blob.        |
| Skeuomorphism | UI mimics real-world materials/objects.                                                              | Use selectively for playful, themed interfaces or controls where the metaphor improves understanding.                |
| Neobrutalism  | Hard borders, strong contrast, chunky type, offset shadows, intentionally raw geometry.              | Hackathons, developer tools, portfolios and youth-oriented products where visual distinctiveness matters.            |
| Claymorphism  | Soft 3D/card-like surfaces, rounded forms, gentle shadows.                                           | Friendly dashboards, education, productivity and playful products.                                                   |
| Minimalism    | Reduction of decoration; hierarchy comes from typography, spacing, alignment and contrast.           | Almost everywhere. Especially useful when the product itself is complex.                                             |
| Liquid Glass  | Fluid/translucent/glassy visual language; often combined with blur, gradients and animated surfaces. | Premium landing pages and visual demos. Use restraint because heavy effects can destroy readability and performance. |

Hackathon shortcut: Pick one visual system and enforce it globally. For example: 12px/24px spacing rhythm + one radius family + one shadow recipe + one typography scale. A consistent mediocre-looking UI beats twelve unrelated styles fighting each other.

## 2. Frontend Libraries — Keep These in Your Toolbox

| Resource           | Reference                                                                                                     | Use it for                                                                                                                                             |
|--------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Floating UI        | <a href="https://floating-ui.com/">https://floating-ui.com/</a>                                               | Tooltips, popovers, dropdowns, dialogs, comboboxes and other floating elements. It is the evolution of Popper 2.                                       |
| Floating UI React  | <a href="https://floating-ui.com/docs/react">https://floating-ui.com/docs/react</a>                           | React primitives for positioning and interactions. Use when you need robust anchoring/collision handling instead of hand-written absolute positioning. |
| AOS                | <a href="https://michalsnik.github.io/aos/">https://michalsnik.github.io/aos/</a>                             | Simple animate-on-scroll effects. Good for fast marketing pages and prototypes.                                                                        |
| Chart.js           | <a href="https://www.chartjs.org/docs/">https://www.chartjs.org/docs/</a>                                     | Charts and dashboards: line, bar, pie/doughnut, radar and related visualizations.                                                                      |
| Luxon              | <a href="https://moment.github.io/luxon/">https://moment.github.io/luxon/</a>                                 | Date/time parsing, formatting, timezone and interval work. Useful in scheduling, dashboards and event applications.                                    |
| SweetAlert2        | <a href="https://sweetalert2.github.io/">https://sweetalert2.github.io/</a>                                   | Polished alerts, confirmation dialogs, forms and notifications without building every modal from scratch.                                              |
| Motion for React   | <a href="https://motion.dev/docs/react">https://motion.dev/docs/react</a>                                     | Production-grade React animations, layout transitions, gestures and scroll animation.                                                                  |
| GSAP ScrollTrigger | <a href="https://gsap.com/docs/v3/Plugins/ScrollTrigger/">https://gsap.com/docs/v3/Plugins/ScrollTrigger/</a> | Advanced scroll-linked animation: trigger, scrub, pin, snap, timelines and progress callbacks.                                                         |

### Library selection rule

Use the smallest tool that solves the problem. AOS is excellent when you want a few entrance animations. Motion is better when the UI itself is interactive and state-driven. GSAP + ScrollTrigger is the heavyweight choice for cinematic, timeline-controlled pages. Native CSS scroll-driven animation is increasingly viable when you do not need a JavaScript animation engine.

### Floating UI mental model

```
npm install @floating-ui/react
```

```
import {useFloating, autoUpdate} from '@floating-ui/react';
```

```
const {refs, floatingStyles} = useFloating({  
  whileElementsMounted: autoUpdate,  
});
```

Use Floating UI when a dropdown/tooltip must remain anchored correctly while scrolling/resizing and must avoid overflowing the viewport.

### Chart.js quick-start mental model

```
import { Chart } from 'chart.js/auto';
```

```
new Chart(canvasElement, {  
  type: 'bar',
```

```
data: {  
  labels: ['A', 'B', 'C'],  
  datasets: [{ label: 'Score', data: [12, 19, 8] }]  
}  
});
```

SweetAlert2 is especially useful for destructive-action confirmation, success/failure feedback, and quick forms. For core application UX, do not hide important information inside alerts.

## 3. Scroll and Motion Engineering

The reels mention parallax, text reveals, scroll-triggered animation, sticky sections, horizontal scroll and progress indicators. The key is to understand the primitives underneath rather than memorizing one library's syntax.

| Resource                       | Reference                                                                                                                                                                       | Use it for                                                                                                               |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| MDN — Scroll-driven animations | <a href="https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Scroll-driven_animations">https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Scroll-driven_animations</a> | Primary reference for native CSS scroll timelines, animation-range, scroll(), view(), scroll-timeline and view-timeline. |
| MDN — IntersectionObserver     | <a href="https://developer.mozilla.org/en-US/docs/Web/API/IntersectionObserver">https://developer.mozilla.org/en-US/docs/Web/API/IntersectionObserver</a>                       | Best primitive for detecting when elements enter/leave the viewport without manually polling scroll position.            |
| GSAP ScrollTrigger             | <a href="https://gsap.com/docs/v3/Plugins/ScrollTrigger/">https://gsap.com/docs/v3/Plugins/ScrollTrigger/</a>                                                                   | Use for complex trigger/scrub/pin/snap sequences and cinematic pages.                                                    |
| Motion                         | <a href="https://motion.dev/docs/react">https://motion.dev/docs/react</a>                                                                                                       | Use in React for declarative animation, gestures, layout transitions and scroll hooks.                                   |
| AOS                            | <a href="https://michalsnik.github.io/aos/">https://michalsnik.github.io/aos/</a>                                                                                               | Fast, simple reveal-on-scroll effects.                                                                                   |

### 3.1 Parallax scrolling

Concept: different visual layers move at different rates. The simplest reliable implementation uses transforms rather than changing layout properties.

```
// Conceptual JS approach
window.addEventListener('scroll', () => {
  const y = window.scrollY;
  layer.style.transform = `translateY(${y * 0.25}px)`;
});
```

Better: For serious pages, use requestAnimationFrame, CSS scroll-driven animation, Motion, or GSAP rather than doing heavy work directly inside every scroll event.

### 3.2 Text reveal on scroll

Common recipe: split text into lines/words/characters, hide with clip-path/transform, then animate each unit as its view progress changes. For accessible production text, keep the actual text in the DOM and avoid making the animation the only way to perceive content.

```
/* Native CSS direction */
.reveal {
  animation: reveal linear both;
  animation-timeline: view();
  animation-range: entry 20% cover 40%;
}

@keyframes reveal {
  from { opacity: 0; transform: translateY(1.2em); }
  to { opacity: 1; transform: translateY(0); }
}
```

### 3.3 Scroll-triggered animation

Use IntersectionObserver for simple 'enter viewport → add class' behavior. Use GSAP ScrollTrigger or Motion when animation progress must be tied continuously to scrolling.

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
```

```

    entry.target.classList.add('is-visible');
  }
});
}, { threshold: 0.15 });

document.querySelectorAll('.reveal').forEach(el => observer.observe(el));

```

### 3.4 Sticky sections

```

.story {
  display: grid;
  grid-template-columns: 1fr 1fr;
}

.story__visual {
  position: sticky;
  top: 5rem;
  height: fit-content;
}

```

Use sticky when one side should remain visible while the other side explains a sequence. This is excellent for product storytelling and feature walkthroughs.

### 3.5 Horizontal scroll

```

.horizontal {
  display: flex;
  gap: 2rem;
  overflow-x: auto;
  scroll-snap-type: x mandatory;
}

.horizontal > * {
  flex: 0 0 min(80vw, 600px);
  scroll-snap-align: center;
}

```

For a true vertical-scroll-to-horizontal cinematic effect, GSAP ScrollTrigger is usually easier than manually rewriting the scroll model.

### 3.6 Scroll progress indicator

```

const progress = document.querySelector('.progress');

function update() {
  const max = document.documentElement.scrollHeight - innerHeight;
  progress.style.transform = `scaleX(${scrollY / max})`;
}

addEventListener('scroll', update, { passive: true });

```

Native CSS can also drive progress indicators with scroll timelines. Prefer native CSS when browser support and fallback requirements are acceptable.

## 4. Layout Patterns — What They Mean and When to Use Them

| Pattern             | Meaning                                                                                                    | Best fit                                                                     |
|---------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| F-pattern           | Text-heavy pages where users scan across the top and then progressively shorter horizontal regions.        | Docs, articles, admin content, information pages.                            |
| Asymmetric          | Elements do not mirror each other; hierarchy is created through scale, whitespace and offset.              | Portfolios, landing pages, creative/product sites.                           |
| Masonry             | Cards of variable height flow into columns with minimized vertical gaps.                                   | Image galleries, feeds, project showcases.                                   |
| Bento grid          | A modular grid with different card sizes arranged into a coherent visual system.                           | Dashboards, product feature pages, portfolio hero sections.                  |
| Retro/Y2K           | Visual language using early-web/2000s motifs, chrome, pixels, gradients, chunky type and playful graphics. | Creative portfolios, campaigns, experimental hackathon demos.                |
| Hero + feature grid | Large value proposition first, then a grid of benefits/features.                                           | SaaS, products, startups, hackathon demos.                                   |
| Sidebar + content   | Persistent navigation/filter/control area beside primary content.                                          | Dashboards, admin panels, docs, data-heavy applications.                     |
| Sticky sections     | A section or element stays fixed while adjacent content progresses.                                        | Storytelling, product walkthroughs, comparison pages.                        |
| Full-page sections  | Large viewport-height sections with strong transitions between them.                                       | Presentations, product launches, visual portfolios. Use carefully on mobile. |

### Bento grid implementation

```
.bento {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  grid-auto-rows: 160px;  
  gap: 1rem;  
}  
  
.card-wide { grid-column: span 2; }  
.card-tall { grid-row: span 2; }
```

### Masonry note

CSS Grid is ideal for explicit card placement. If you need true variable-height masonry with minimal gaps, evaluate CSS masonry support in your target browsers or use a masonry library. Do not assume every browser supports the same experimental masonry APIs.

Responsive rule: design the mobile composition separately. Do not merely shrink a desktop layout. A bento grid often becomes a single-column or two-column sequence; a horizontal showcase may become a swipeable carousel.

## 5. AI Frontend Design + Coding Stack

The reels suggest combining AI design generation with AI coding and an agentic development environment. The practical workflow is more important than the branding.

| Resource             | Reference                                                                                                                                                                                 | Use it for                                                                                                       |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Google Stitch        | <a href="https://stitch.withgoogle.com/">https://stitch.withgoogle.com/</a>                                                                                                               | Generate/refine UI concepts from prompts and visual inputs; useful for rapid frontend ideation.                  |
| Google Stitch update | <a href="https://blog.google/innovation-and-ai/models-and-research/google-labs/stitch-updates/">https://blog.google/innovation-and-ai/models-and-research/google-labs/stitch-updates/</a> | Official description of Stitch's 2026 workflow, including design iteration and export toward Google Antigravity. |
| Claude / Claude Docs | <a href="https://docs.claude.com/">https://docs.claude.com/</a>                                                                                                                           | Claude developer docs, Claude Code, API and implementation references.                                           |
| Claude Code          | <a href="https://docs.claude.com/en/docs/claude-code/overview">https://docs.claude.com/en/docs/claude-code/overview</a>                                                                   | Agentic coding workflow for repository-level changes, debugging and development tasks.                           |
| CodeRabbit           | <a href="https://www.coderabbit.ai/">https://www.coderabbit.ai/</a>                                                                                                                       | AI-assisted code review for pull requests.                                                                       |

### Recommended hackathon workflow

Step 1 — Product: write the user, problem, constraints and success metric.

Step 2 — Design: use Stitch or a component library to explore 2–3 directions.

Step 3 — System: decide typography, spacing, colors, radii, components and responsive rules before coding.

Step 4 — Build: use Claude Code/your preferred coding agent to implement the agreed design.

Step 5 — Motion: add one coherent motion system rather than random animations.

Step 6 — Review: run accessibility, responsiveness, performance and code review before polishing.

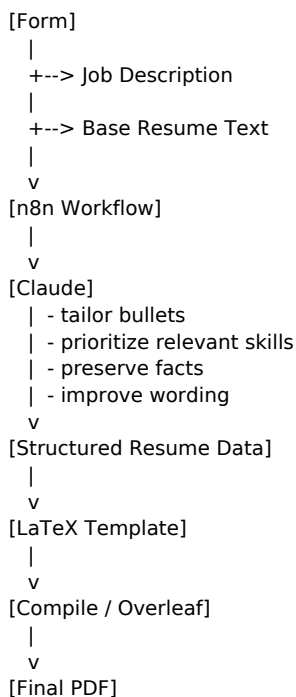
Do not let the AI continuously redesign the application while you are implementing it. That is how a 24-hour hackathon becomes a 24-hour hostage situation.

## 6. Automated AI Resume Generator — n8n + Claude + Overleaf

This was one of the strongest automation ideas in the supplied material. The concept is to turn a base resume and a job description into a tailored LaTeX/PDF resume with minimal manual work.

| Resource      | Reference                                                           | Use it for                                                                     |
|---------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------|
| n8n Docs      | <a href="https://docs.n8n.io/">https://docs.n8n.io/</a>             | Workflow automation, integrations, credentials, AI nodes, APIs and deployment. |
| n8n Academy   | <a href="https://learn.n8n.io/">https://learn.n8n.io/</a>           | Hands-on learning paths for workflows, integrations, AI and best practices.    |
| Overleaf Docs | <a href="https://docs.overleaf.com/">https://docs.overleaf.com/</a> | LaTeX projects, compilation, project files and collaboration.                  |
| Claude Docs   | <a href="https://docs.claude.com/">https://docs.claude.com/</a>     | API and Claude Code documentation for AI-powered text processing.              |

### Architecture



### Critical prompt constraints

The AI should never invent achievements, technologies, employers, dates, metrics or certifications. Tell it to preserve factual integrity and only rephrase/reorder information present in the source resume. Separate optimization from factual enrichment.

### Suggested structured output

```
{
  "summary": "...",
  "skills": ["..."],
  "experience": [
    {
      "company": "...",
      "role": "...",
      "bullets": ["...", "..."]
    }
  ]
}
```

```
],  
  "projects": ["..."],  
  "keywords_used": ["..."]  
}
```

Hackathon extension: add a JD keyword analysis stage before Claude rewriting. Then produce a final ATS-risk report showing missing keywords, suspicious repetition, and unsupported claims.

## 7. Problem Statements + Freelancing

The supplied reels mention FixMyI tch/Razorpay and three freelance platforms. These belong in your opportunity-finding toolkit rather than your frontend toolkit.

| Resource      | Reference                                                                   | Use it for                                                                                                                                                          |
|---------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Razorpay      | <a href="https://razorpay.com/">https://razorpay.com/</a>                   | Official company reference. The reel specifically mentions a problem-statement portal associated with Razorpay; verify the current portal URL before relying on it. |
| Contra        | <a href="https://contra.com/">https://contra.com/</a>                       | Freelance jobs, profiles and independent-work infrastructure. Useful for finding remote projects and building a professional presence.                              |
| PeoplePerHour | <a href="https://www.peopleperhour.com/">https://www.peopleperhour.com/</a> | Freelance marketplace with fixed-price offers and project-based work, including technology and programming.                                                         |
| Toptal        | <a href="https://www.toptal.com/">https://www.toptal.com/</a>               | Premium freelance talent network with a strong screening/vetting model; better viewed as a longer-term target than a quick gig marketplace.                         |

### Freelancing checklist

1. Pick one sellable service rather than 'I can do everything.'
2. Build 2-3 polished examples.
3. Package the outcome: e.g. 'responsive SaaS landing page in React' rather than 'frontend development.'
4. Show before/after or measurable outcomes.
5. Use the platforms as lead sources; do not depend on one marketplace.

### Problem-statement strategy for hackathons

Prefer problems where the pain is concrete, the user is identifiable, a demo can show the core value in under 60 seconds, and the solution can be scoped to a weekend. A problem portal is useful because it gives you a real constraint instead of forcing your team to invent a fake problem.

## 8. Claude 'Secret Codes' — What to Keep and What to Doubt

The reel claims that five strings make Claude dramatically better. These should be treated as prompting ideas, not secret system-level unlocks.

| Prompt    | Claim                                                                  | Practical verdict                                                                                                                                 |
|-----------|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| /ghost    | Claimed humanization / AI-detector evasion.                            | VERIFY / do not rely on it as an official feature. Better: specify tone, sentence variation, audience, evidence and editing constraints directly. |
| ARTIFACTS | Create working apps/games/dashboards in Claude's artifact environment. | Real capability category, but the keyword itself is not a magic multiplier.                                                                       |
| OODA      | Use Observe–Orient–Decide–Act as a decision framework.                 | Useful as an explicit reasoning framework; tell the model what output structure you want.                                                         |
| L99       | Claimed advanced expert mode.                                          | VERIFY. No reason to treat this as a guaranteed hidden mode.                                                                                      |
| /godmode  | Claimed aggressive/comprehensive mode.                                 | VERIFY. Better to specify depth, assumptions, constraints, tests and deliverables.                                                                |

### Better 'expert mode' prompt

Act as a senior frontend engineer.

Before coding:

1. state assumptions,
2. identify constraints,
3. propose the simplest robust architecture,
4. flag accessibility/performance risks,
5. implement only after the plan is clear,
6. provide tests and edge cases,
7. do not invent APIs or facts.

This is much more reliable than hunting for secret words that allegedly unlock a boss fight inside a chatbot.

## 9. Antigravity + Claude Code + Agentic Coding

The reels mention Antigravity with tools/workflows such as GSD, Ralph Loop and CodeRabbit. The useful concept is an agentic development loop: plan → execute → test → review → iterate.

| Resource                             | Reference                                                                                                                                                                                 | Use it for                                                                                             |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Google Antigravity / Stitch workflow | <a href="https://blog.google/innovation-and-ai/models-and-research/google-labs/stitch-updates/">https://blog.google/innovation-and-ai/models-and-research/google-labs/stitch-updates/</a> | Google describes exporting Stitch designs into Google Antigravity for continued development.           |
| GSD — GitHub search                  | <a href="https://github.com/search?q=get+shit+done+claude+code&amp;type=repositories">https://github.com/search?q=get+shit+done+claude+code&amp;type=repositories</a>                     | Reference point for the GSD workflow mentioned in the reel. Exact implementation varies by repository. |
| Ralph Loop — GitHub search           | <a href="https://github.com/search?q=ralph+loop+claude+code&amp;type=repositories">https://github.com/search?q=ralph+loop+claude+code&amp;type=repositories</a>                           | Reference point for iterative agent loops. Verify the specific repository before adopting it.          |
| CodeRabbit                           | <a href="https://www.coderabbit.ai/">https://www.coderabbit.ai/</a>                                                                                                                       | Automated review layer for pull requests and code quality feedback.                                    |
| Claude Code docs                     | <a href="https://docs.claude.com/en/docs/claude-code/overview">https://docs.claude.com/en/docs/claude-code/overview</a>                                                                   | Official Claude Code implementation/reference material.                                                |

### A safe agent loop

```
PLAN
↓
IMPLEMENT
↓
RUN TESTS / BUILD
↓
INSPECT DIFF
↓
CODE REVIEW
↓
FIX
↓
REPEAT UNTIL ACCEPTANCE CRITERIA PASS
```

For hackathons, keep an explicit acceptance checklist in the repository. The agent should be judged against that checklist, not against whether its output 'looks impressive'.

Do not give an agent unrestricted destructive permissions in a project containing secrets or production credentials. Use environment variables, least privilege and version control.

# 10. Hackathon Implementation Playbook

This section turns the collected resources into an actual workflow you can execute under time pressure.

## 0-60 minutes: Decide

- Define one user and one painful problem.
- Write the demo sentence: 'For X, we reduce Y by doing Z.'
- Pick the smallest end-to-end flow that proves the idea.
- Choose one visual style and one motion style.
- Create acceptance criteria before coding.

## 60-180 minutes: Design + skeleton

- Use Stitch or Ulverse for visual exploration.
- Create the layout skeleton first: hero, navigation, main interaction, result state.
- Create reusable components before filling the page.
- Choose Motion/GSAP/AOS only after identifying the required animation complexity.

## 3-8 hours: Build the vertical slice

- Make one complete user journey work end-to-end.
- Use Floating UI for robust popovers/tooltips instead of reinventing positioning.
- Use Chart.js for standard data visualizations.
- Use Luxon for date/time logic instead of hand-rolling timezone calculations.
- Use SweetAlert2 only where a modal/confirmation actually improves the UX.
- Keep API calls, loading states, errors and empty states visible in the design.

## 8-16 hours: Make it convincing

- Add responsive behavior.
- Add a small number of high-impact animations.
- Use a coherent spacing/radius/typography system.
- Add one standout interaction: sticky story, scroll reveal, bento transition, horizontal showcase, etc.
- Run through the product as a first-time user and remove friction.

## Final hours: Demo hardening

- Test the happy path from a clean session.
- Test missing data and API failures.
- Check mobile width.
- Check keyboard navigation for important controls.
- Remove console errors.
- Confirm the deployed URL and demo data work.
- Prepare a 60-second demo script: problem → action → result → impact.

## The 60-second demo structure

0-10s Problem: who suffers and why?

10-20s Solution: what did you build?  
20-45s Live demo: show the core journey.  
45-55s Differentiator: what makes it better?  
55-60s Impact: measurable / believable outcome.

The judge does not care that you used seventeen libraries. They care that the thing works, the problem matters, and the team can explain why its architecture makes sense.

# 11. Quick-Reference Decision Table

| Situation                                 | First tool/resource to reach for                                 |
|-------------------------------------------|------------------------------------------------------------------|
| Need a component fast                     | Uiverse                                                          |
| Need tooltip/popover/dropdown positioning | Floating UI                                                      |
| Simple reveal-on-scroll                   | AOS or IntersectionObserver                                      |
| Complex cinematic scroll sequence         | GSAP ScrollTrigger                                               |
| React animation / layout / gestures       | Motion                                                           |
| Native scroll-linked animation            | CSS scroll-driven animations                                     |
| Dashboard charts                          | Chart.js                                                         |
| Date/time/timezone logic                  | Luxon                                                            |
| Fast polished confirmation/modal          | SweetAlert2                                                      |
| AI-generated UI concept                   | Google Stitch                                                    |
| AI repository-level coding                | Claude Code                                                      |
| Automate multi-step workflows             | n8n                                                              |
| AI-tailored resume                        | n8n + Claude + LaTeX/Overleaf                                    |
| Find freelance work                       | Contra / PeoplePerHour / Toptal                                  |
| Find real-world project problems          | Problem portals such as the Razorpay example; verify current URL |

## Official reference URLs collected

<https://uiverse.io/>

<https://floating-ui.com/>

<https://michalsnik.github.io/aos/>

<https://www.chartjs.org/docs/>

<https://moment.github.io/luxon/>

<https://sweetalert2.github.io/>

<https://motion.dev/docs/react>

<https://gsap.com/docs/v3/Plugins/ScrollTrigger/>

[https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Scroll-driven\\_animations](https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Scroll-driven_animations)

<https://developer.mozilla.org/en-US/docs/Web/API/IntersectionObserver>

<https://stitch.withgoogle.com/>

<https://blog.google/innovation-and-ai/models-and-research/google-labs/stitch-updates/>

<https://docs.claude.com/>

<https://docs.n8n.io/>

<https://learn.n8n.io/>

<https://docs.overleaf.com/>

<https://contra.com/>

<https://www.peopleperhour.com/>

<https://www.toptal.com/>

<https://www.coderabbit.ai/>

## Source-quality note

Where possible, this guide links directly to official documentation or the official project site. Items that originated only as reel claims without an identifiable canonical resource are marked VERIFY or described as a concept rather than promoted as a fact. Re-check rapidly changing tools before using them in production.

End of current collection — future reels can be folded into the same resource-vault structure.